



PONTIFICIA UNIVERSIDAD CATÓLICA DE CHILE
ESCUELA DE INGENIERÍA
DEPARTAMENTO DE CIENCIA DE LA COMPUTACIÓN

IIC2213 — Lógica para Ciencia de la Computación — 2024-01

TAREA 4

Publicación: Jueves 16 de mayo.

Entrega: **Martes 04 de junio hasta las 23:59 horas.**

Indicaciones

- La tarea es **individual**. Puede discutir con sus compañera/os o utilizar información externa (libros, foros, páginas web, etc...), siempre y cuando usted mismo escriba su tarea, con sus propias palabras. Debe **citar** cualquier referencia que utilice. Se considerará copia: usar soluciones de compañera/os (con o sin consentimiento del plagiado), entregar su solución para que la vea otra/o compañera/o, usar fuentes externas sin citar.
- Cada pregunta tiene una nota entre 1 y 7 (hay un punto base). Los items dentro de una pregunta valen lo mismo. La nota de la tarea es el promedio de las notas de las preguntas.
- La tarea **debe** ser escrita en $\text{\LaTeX}$. No se aceptarán tareas escritas de otra forma. No olvide indicar su nombre, RUT y número de alumna/o.
- Se aceptan atrasos. Se descontará 1 punto por día de atraso, sobre la nota final. Se permite como máximo 3 días de atraso.

Pregunta 1

1. Demuestre que la clase P es cerrada bajo unión, intersección y complemento. Es decir, demuestre que si $L, L' \in P$, entonces $L \cup L' \in P$, $L \cap L' \in P$, $\overline{L} \in P$.
2. ¿Es cierto que todo lenguaje finito está en P ? Argumente su respuesta.

Solución

Parte 1

Sean $L, L' \in P$, entonces existen máquinas de Turing deterministas M y M' tales que deciden los lenguajes en tiempo polinomial. Luego podemos construir una nueva máquina $M_{\cup}$ de dos cintas tal que

1. Dado un input w , lo copia en la segunda cinta.
2. Se ejecuta M en la primera cinta, si se acepta, entonces la máquina acepta el input.
3. Si se rechazó en el paso anterior, se procede a ejecutar M' en la segunda cinta y nuevamente, si se acepta, la máquina acepta el input.
4. Si no se aceptó en ninguna de las dos ejecuciones, se rechaza

Claramente el tiempo de ejecución de esta máquina será la suma del tiempo de ejecución de copiar el input, el tiempo que toma M y el tiempo que toma M' , al ser todo esto polinomial, tenemos que la suma también será polinomial. Además, pasar de una máquina de dos cintas a una equivalente de una cinta requiere tiempo a lo más cuadrático, por lo que podemos construir una máquina equivalente $M'_\cup$ de una cinta que acepte el mismo lenguaje que $M_\cup$ en tiempo polinomial.

Finalmente, notemos que el lenguaje aceptado por la máquina serán las palabras que son aceptadas por M o por M' , es decir, $L(M'_\cup) = L(M) \cup L(M') = L \cup L'$. Por lo tanto, $L \cup L' \in P$

Por otro lado, dados lenguajes $L, L' \in P$ como antes, podemos construir una máquina $M_\cap$ de dos cintas de la siguiente manera

1. Dado un input w , lo copia en la segunda cinta.
2. Se ejecuta M en la primera cinta, si se rechaza, entonces la máquina rechaza el input.
3. Si se aceptó en el paso anterior, se procede a ejecutar M' en la segunda cinta y, si se acepta, la máquina acepta el input.
4. Si la segunda máquina no acepta, se rechaza el input.

Por el mismo argumento anterior, sabemos que existe una máquina de una cinta $M'_\cap$ que acepta el mismo lenguaje de $M_\cap$ en tiempo polinomial. Además, $M_\cap$ acepta las palabras que sean aceptadas por M y por M' , es decir $L(M'_\cap) = L(M) \cap L(M') = L \cap L'$ y $L \cap L'$ está en P .

Finalmente, tenemos que si L está en P entonces existe una máquina de Turing que lo decide en tiempo polinomial, por lo tanto podemos construir M' de tal manera que dado un input w

1. Se ejecuta M sobre w
2. Si M acepta, se rechaza w
3. Si M rechaza, se acepta w

Esta máquina tendrá el mismo tiempo de ejecución que M , por lo que trabaja en tiempo polinomial y su lenguaje aceptado son todas las palabras rechazadas por M . Es decir, $L(M') = \overline{L(M)} = \overline{L}$, por lo tanto $\overline{L} \in P$.

Parte 2

Sí, todo lenguaje finito L está en P , es posible demostrarlo por inducción en la cantidad de elementos de L

1. Caso base: Si $L = \emptyset$ podemos tomar la máquina que solo tiene estado inicial y no tiene ningún estado de aceptación. Esta máquina no realizará ningún paso y acepta exactamente el vacío, entonces $L \in P$. Si $L = \{w_1 w_2 \dots w_n\}$, entonces construimos una máquina M con estados $\{q_0, q_{w_1}, q_{w_1 w_2}, \dots, q_{w_1 \dots w_n}, q_F\}$ tal que
 - a) Si la máquina recibe un input w , lee el primer símbolo y si es w_1 pasa a estado q_{w_1} .
 - b) Para cada $i \in \{1, \dots, n\}$, si la cabeza está en estado $q_{w_1 \dots w_{i-1}}$ y lee w_i , entonces pasa a estado $q_{w_1 \dots w_i}$ y se mueve a la derecha.
 - c) Se repite el paso anterior todas las veces que se pueda, si se llega a estado $q_{w_1 \dots w_n}$ y se lee vacío, entonces se pasa a q_F y se acepta la palabra.

Esta máquina realiza a lo más n pasos, por lo que su lenguaje aceptado está en P . Además, aceptará solo la palabra $w_1 \dots w_n$, es decir, acepta exactamente L , entonces $L \in P$.

2. Paso inductivo: Si L es un lenguaje de cardinalidad n , entonces tenemos que $L = L' \cup \{w\}$ con L' un lenguaje de cardinalidad $n - 1$. Por hipótesis inductiva sabemos que $L' \in P$ y por caso base sabemos que $\{w\} \in P$, entonces por la parte 1 de este ejercicio podemos concluir que $L = L' \cup \{w\}$ está en P .

Parte 1: Se da 1 punto por cada uno de los lenguajes, 0.5 puntos por construir la máquina y otros 0.5 puntos por la justificación.

Parte 2: 1 punto por decir que la afirmación es verdadera y 2 puntos por la explicación. También considerar correcto si en lugar de inducción, se construye una máquina que acepta lenguajes finitos.

Pregunta 2

1. Demuestre que la clase NP es cerrada bajo unión. Es decir, si $L, L' \in \text{NP}$, entonces $L \cup L' \in \text{NP}$.
2. Demuestre que la clase NP es cerrada bajo intersección. Es decir, si $L, L' \in \text{NP}$, entonces $L \cap L' \in \text{NP}$.

Solución

Parte 1

Sean L, L' lenguajes en NP entonces tenemos polinomios $p, q : \mathbb{N} \rightarrow \mathbb{N}$ y máquinas de Turing M, M' polinomiales tales que

$$\begin{aligned} w \in L &\iff \exists x \in A^* \text{ tal que } |x| \leq p(|w|) \text{ y } M \text{ acepta } (w, x) \\ w \in L' &\iff \exists x \in A^* \text{ tal que } |x| \leq q(|w|) \text{ y } M' \text{ acepta } (w, x) \end{aligned}$$

Asumiremos la máquina M rechaza todos los inputs (w, x) tales que $|x| > p(|w|)$ y M' rechaza todos los inputs (w, x) tales que $|x| > q(|w|)$. Esto se puede hacer, ya que toma un tiempo polinomial chequear si un input cumple esta condición. Por la parte 1 de la pregunta 1, sabemos que existe una máquina M'' que acepta $L(M) \cup L(M')$ en tiempo polinomial.

Sea $w \in L \cup L'$, entonces existe $x \in A^*$ tal que $|x| \leq p(|w|)$ y M acepta (w, x) o $|x| \leq q(|w|)$ y M' acepta (w, x) . Notemos que esto significa que $|x| \leq p + q(|w|)$ y (w, x) es aceptado por M'' .

Por otro lado, si existe $x \in A^*$ tal que $|x| \leq p + q(|w|)$ y (w, x) es aceptado por M'' , entonces (w, x) es tal que $|x| \leq p(|w|)$ y es aceptado por M o $|x| \leq q(|w|)$ y es aceptado por M' . Entonces $w \in L$ o $w \in L'$, es decir $w \in L \cup L'$.

Por lo tanto

$$w \in L \cup L' \iff \exists x \in A^* \text{ tal que } |x| \leq p + q(|w|) \text{ y } M'' \text{ acepta } (w, x)$$

Como $p + q$ es un polinomio, tenemos que $L \cup L'$ está en NP.

Parte 2

Sean L, L' lenguajes en NP entonces tenemos polinomios $p, q : \mathbb{N} \rightarrow \mathbb{N}$ y máquinas de Turing M, M' polinomiales tales que

$$\begin{aligned} w \in L &\iff \exists x \in A^* \text{ tal que } |x| \leq p(|w|) \text{ y } M \text{ acepta } (w, x) \\ w \in L' &\iff \exists x \in A^* \text{ tal que } |x| \leq q(|w|) \text{ y } M' \text{ acepta } (w, x) \end{aligned}$$

Ahora podemos construir máquinas N y N' tales que

$$\begin{aligned} (w, x\#y) \in L(N) &\iff (w, x) \in L(M), |x| \leq p(|w|) \text{ y } |y| \leq q(|w|) \\ (w, x\#y) \in L(N') &\iff (w, y) \in L(M'), |x| \leq p(|w|) \text{ y } |y| \leq q(|w|) \end{aligned}$$

Estas máquinas siguen siendo polinomiales, ya que computar los polinomios y chequear el largo del input se pueden hacer en tiempo polinomial entonces agregar eso a las ejecuciones de M y M' sigue siendo polinomial. Por la parte 1 de la pregunta 1, sabemos que existe una máquina M'' que acepta $L(M) \cap L(M')$ en tiempo polinomial.

Sea $w \in L \cap L'$, entonces existe $x \in A^*$ tal que $|x| \leq p(|w|)$ y M acepta (w, x) y existe $y \in A^*$ tal que $|y| \leq q(|w|)$ y M' acepta (w, y) . Notemos que esto significa que $|x\#y| \leq p(|w|) + q(|w|) + 1$ y $(w, x\#y)$ es aceptado por M'' .

Por otro lado, si existe $x, y \in A^*$ tales que $(w, x\#y)$ es aceptado por M'' , entonces (w, x) es tal que $|x| \leq p(|w|)$ y es aceptado por M y (w, y) es tal que $|y| \leq q(|w|)$ y es aceptado por M' . Entonces $w \in L$ y $w \in L'$, es decir $w \in L \cap L'$.

Por lo tanto

$$w \in L \cap L' \iff \exists z \in A^* \text{ tal que } |z| \leq p(|w|) + q(|w|) + 1 \text{ y } M'' \text{ acepta } (w, z)$$

Entonces, tenemos que $L \cap L'$ está en NP.

En ambas partes se da 1 punto por enunciar correctamente la definición de NP para ambos lenguajes iniciales y 1 punto por cada una de las implicancias de la definición para el nuevo lenguaje.

En caso de demostrar construyendo las máquinas no deterministas, se da un punto por enunciar correctamente la definición de NP con máquinas de Turing, 1 punto por la construcción de la nueva máquina y 1 punto por justificar que el lenguaje aceptado es el correcto.

Pregunta 3

Dada una matriz de enteros A de $n \times m$ y un vector $\vec{b}$ de n enteros, el problema de *programación entera* en su versión de problema de decisión consiste en verificar si existe un vector $\vec{x}$ de m enteros tal que $A\vec{x} \leq \vec{b}$. Por ejemplo, el siguiente es un problema de programación entera para el cual el vector $\vec{x} = (1, 1)$ es una solución:

$$\begin{bmatrix} -1 & 0 \\ 0 & -1 \\ 1 & 1 \end{bmatrix} \cdot \begin{bmatrix} x_1 \\ x_2 \end{bmatrix} \leq \begin{bmatrix} -1 \\ 0 \\ 3 \end{bmatrix}$$

Como lenguaje, lo definimos por

$$\text{PE} = \{(A, \vec{b}) \mid A \text{ de } n \times m, \vec{b} \text{ de } n \times 1 \text{ y existe vector } \vec{x} \text{ de } m \times 1 \text{ tal que } A\vec{x} \leq \vec{b}\}.$$

Demuestre que PE es NP-completo.

(Hint: para el NP-hardness, pruebe que $3\text{SAT} \leq_p \text{PE}$.)

Solución

En primer lugar, notemos que podemos construir una máquina M que dado input $((A, \vec{b}), \vec{x})$ revisa si $A\vec{x} \leq \vec{b}$ en tiempo polinomial, ya que requiere hacer a lo más $|A||\vec{x}|$ multiplicaciones y sumas y luego comparar $|\vec{b}|$ valores. Por lo tanto,

$$(A, \vec{b}) \in \text{PE} \iff \exists \vec{x} \text{ tal que } |x| \leq p(|(A, \vec{b})|) \text{ y } M \text{ acepta } (w, x)$$

Sabemos que el polinomio p existe, ya que el verificador tendrá a lo más tantas coordenadas como A y la codificación binaria de cada una de estas coordenadas también será de largo polinomial sobre $|(A, \vec{b})|$, ya que la solución no puede entradas arbitrariamente grandes, depende de las entradas de $\vec{b}$.

Es decir, PE está en NP.

Para probar que $3\text{SAT} \leq_p \text{PE}$, tendremos que construir una función computable $f : \{0, 1\}^* \rightarrow \{0, 1\}^*$ tal que

$$w \in 3\text{SAT} \iff f(w) \in \text{PE}$$

Claramente, estamos asumiendo que usamos alguna codificación para los elementos de ambos lenguajes.

Para esto, supongamos que $w \in \{0, 1\}^*$. Si w no es la codificación de una fórmula en 3-CNF, entonces $f(w) = 0$. Si w es la codificación de una fórmula en 3-CNF con n variables y m cláusulas, entonces $f(w)$ será la codificación de $(A, \vec{b})$, donde

1. A es una matriz $(2n + m) \times n$ tal que las primeras n filas son la matriz identidad $I_{n \times n}$, las segundas n filas son la matriz $-I_{n \times n}$ y las últimas m filas son tales que cada una representa una cláusula de la siguiente manera: si la variable i -ésima aparece negada, hay un 1 en la posición i y si aparece normal, hay un -1 en la posición i , el resto se rellena con 0's.
2. $\vec{b}$ es un vector de $2n + m$ variables tales que las primeras n variables son 1's, las segundas n variables son 0's y las últimas m variables son la cantidad de variables que aparecen negadas en cada cláusula menos 1 (en el mismo orden que en A).

Para que quede claro el funcionamiento de f se presenta un ejemplo, sea w la codificación de

$$\varphi = (w_1 \vee w_2 \vee \neg w_3) \wedge (\neg w_2 \vee \neg w_1 \vee w_4) \wedge (w_2 \vee w_3 \vee w_4)$$

Entonces el sistema representado por $f(w)$ se vería de la siguiente manera,

$$\begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \\ -1 & 0 & 0 & 0 \\ 0 & -1 & 0 & 0 \\ 0 & 0 & -1 & 0 \\ 0 & 0 & 0 & -1 \\ -1 & -1 & 1 & 0 \\ 1 & 1 & 0 & -1 \\ 0 & -1 & -1 & -1 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ x_3 \\ x_4 \end{bmatrix} \leq \begin{bmatrix} 1 \\ 1 \\ 1 \\ 1 \\ 0 \\ 0 \\ 0 \\ 0 \\ 0 \\ 1 \\ -1 \end{bmatrix}$$

La idea aquí es que las últimas 3 desigualdades chequean si se están satisfaciendo todas las cláusulas. Por ejemplo, para la primera cláusula agregamos $-x_1 - x_2 + x_3 \leq (\# \text{variables negadas}) - 1 = 0$, esto quiere decir que una valuación va a ser aceptada solo cuando $x_1 = 1$, $x_2 = 1$ o $x_3 = 0$, es decir, solo cuando se satisfaga $(w_1 \vee w_2 \vee \neg w_3)$.

Ahora probemos que f es reducción. Claramente es computable, ya que es prácticamente un cambio de formato del input. Además, sea $w \in \{0, 1\}^*$, notemos que si w no es codificación de una fórmula 3-CNF, entonces $w \notin 3\text{SAT}$ y $f(w) = 0 \notin \text{PE}$.

Ahora supongamos que w es codificación de una fórmula φ 3-CNF. Si tuviéramos una solución x para el sistema relacionado a $f(w)$, entonces por la fórmula que construimos A este debe ser un vector de 0's y 1's, por lo que podríamos asignar a las variables de φ la valuación que hace w_i verdadero si la i -ésima coordenada de x es 1 y la hace falsa si no. Esta valuación satisface φ , por la forma que construimos las últimas filas, ya que si tenemos un literal verdadero en una cláusula, entonces al hacer el producto con las variables, esto será menor que el caso en el que no se cumple ningún literal, es decir, será menor que la suma de variables negadas.

Si w es la codificación de una fórmula φ 3-CNF tal que es satisfacible, entonces notemos que el sistema tendrá como solución el vector tal que la i -ésima coordenada es la valuación de la i -ésima variable. Esto se obtiene de forma similar a lo descrito anteriormente.

Por lo tanto, $w \in 3\text{SAT} \Leftrightarrow f(w) \in \text{PE}$, es decir $3\text{SAT} \leq_p \text{PE}$.

Si vemos el ejemplo anterior, notemos que la valuación σ que hace todas las variables 1, menos w_2 es una solución para el sistema, por lo tanto tenemos que,

$$\begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \\ -1 & 0 & 0 & 0 \\ 0 & -1 & 0 & 0 \\ 0 & 0 & -1 & 0 \\ 0 & 0 & 0 & -1 \\ -1 & -1 & 1 & 0 \\ 1 & 1 & 0 & -1 \\ 0 & -1 & -1 & -1 \end{bmatrix} \begin{bmatrix} 1 \\ 0 \\ 1 \\ 1 \end{bmatrix} \leq \begin{bmatrix} 1 \\ 1 \\ 1 \\ 1 \\ 0 \\ 0 \\ 0 \\ 0 \\ 0 \\ 1 \\ -1 \end{bmatrix}$$

Como 3SAT es NP-completo, podemos concluir que PE es NP-completo.

Se dan 2 puntos por probar que PE está en NP (con cualquiera de las dos definiciones) y 4 puntos por la reducción de 3SAT, 2 por la construcción y 2 por la argumentación.